LEARNING MATERIAL

UNIT-III

REPRESENTATION OF KNOWLEDGE

Syllabus:

Representation of Knowledge: Knowledge representation issues, predicate logic- logic programming, semantic nets- frames and inheritance, representing knowledge using rules. Reasoning under uncertainty, review of probability, Bayes' probabilistic interferences and dempster shafer theory.

Learning Outcomes:

At the end of the unit, students will be able to

- work knowledge bases that combine logical and semantic nets.
- describe probabilistic representations.

Module 1: Knowledge representation and predicate logic

Module 2: Semantic nets- frames and inheritance

Module 3: Reasoning under uncertainty

1. KNOWLEDGE REPRESENTATION

- Humans are best at understanding, reasoning, and interpreting knowledge.
- Human knows things, as per their knowledge they perform various actions in the real world.
- But how machines do all these things comes under knowledge representation and reasoning. Hence we can describe Knowledge representation.
- Following are the kind of knowledge which needs to be represented in AI systems:
- **Object:** All the facts about objects in our world domain.
- **Events:** Events are the actions which occur in our world.
- **Performance:** It describes behavior which involves knowledge about how to do things.
- **Meta-knowledge:** It is knowledge about what we know.
- **Facts:** Facts are the truths about the real world and what we represent.
- **Knowledge-Base:** The central component of the knowledge-based agents is the knowledge base. It is represented as KB. The Knowledgebase is a group of the Sentences (Here, sentences are used as a technical term and not identical with the English language).

1.1 Knowledge representation issues

- The fundamental goal of Knowledge Representation is to facilitate inference (conclusions) from knowledge.
- The issues that arise while using Knowledge Representation techniques are follows:
  1. Important Attributes
  2. Relationships among Attributes
  3. Choosing the Granularity of Representation
  4. Representing Sets of Objects
  5. Finding the Right Structures as Needed

1.1.1 Important Attributes

- There are two attributes ***instance*** and ***isa*** are of very general significance.
- These attributes are important because they support property inheritance. They are called a variety of things in AI systems, but the names may differ.
- These attributes represent class membership and class inclusion and that class inclusion is transitive.
- In logic-based systems, these relationships may be represented implicitly by a set of predicates describing particular classes.

1.1.2 Relationships among Attributes

- The attributes that we use to describe objects are themselves entities that we represent.
- What properties do they have independent of the specific knowledge they encode?
- There are four such properties that deserve mention here:

  (i) Inverses

  (ii) Existence in an ***isa*** hierarchy

  (iii) Techniques for reasoning about values

  (iv) Single-valued attributes

1.1.3 Choosing the Granularity of Representation

- Regardless of the particular representation formalism we choose, it is necessary to answer the question "At what level of detail should the world be represented?"
- Another way this question is often phrased is "What should be our primitives?" Should there be a small number of low-level ones or should there be a larger number covering a range of granularities?
- A brief example illustrates the problem. Suppose we are interested in the following fact:

John spotted Book.

We could represent this as

spotted(agent(John), object(Book))

Such a representation would make it easy to answer questions such as:

Who spotted Book?

But now suppose we want to know:

Did John see Book?

The obvious answer is "yes," but given only the one fact we have, we cannot discover that answer. We could, of course, add other facts, such as

spotted(x, y) → *saw(x, y)*

1.1.4 Representing Sets of Objects

- It is important to be able to represent sets of objects for several reasons.
- One is that there are some properties that are true of sets that are not true of the individual members of a set.
- As examples, consider the assertions that are being made in the sentences "There are more followers than people in India" and "English speakers can be found all over the world."
- The only way to represent the facts described in these sentences is to attach assertions to the sets representing people, followers, and English speakers, since, for example, no single English speaker can be found all over the world.
- The other reason that it is important to be able to represent sets of objects is that if a property is true of all (or even most) elements of a set, then it is more efficient to associate it once with the set rather than to associate it explicitly with every element of the set.
- There are two ways to state a definition of a set and its elements.
- The first is to list the members. Such a specification is called an *extensional* definition.
- The second is to provide a rule that, when a particular object is evaluated, returns true or false depending on whether the object is in the set or not. Such a rule is called an *intensional* definition.

1.1.5 Finding the Right Structures as Needed

- Given a large amount of knowledge stored in a database, how can relevant parts are accessed when they are needed?
- This is about access to right structure for describing a particular situation.
- This requires, selecting an initial structure and then revising the choice.

2. PREDICATE LOGIC- LOGIC PROGRAMMING

- The logical formalism is appealing because it immediately suggests a powerful way of deriving new knowledge from mathematical deduction. In this formalism, we can conclude that a new statement is true by proving that it follows from the statements that are already known.
- Thus the idea of a proof, as developed in mathematics as a rigorous way of demonstrating the truth of an already believed proposition, can be extended to include deduction as a way of deriving answers to questions and solutions to problems.
- One of the early domains in which AI techniques were explored was mechanical theorem proving, by which was meant proving statements in various areas of mathematics.
- The following are standard logic symbols:

  → (implication), ~ (not), V (or), ∧ (and), ∀ (for all), ∃ (there exists).

2.1 Representing Simple Facts in Logic

- Propositional logic as a way of representing the sort of world knowledge that an AI system might need.
- Propositional logic is simple to deal with and a decision procedure for it exists. We can easily represent real-world facts as logical *propositions* written as *well formed formulas (wff's)* in propositional logic, as shown in Figure 1.
- Using these propositions, we can conclude the facts.

It is raining.

RAINING

It is sunny.

SUNNY

It is windy.

WINDY

If it is raining, then it is not sunny.

RAINING → ~ SUNNY

Figure 1: *Some Simple Facts in Prepositional Logic*

- Predicate means procedure / Relationship / General word.

Another Example:

(i) Charles Babbage is a man.

We could write:

CHARLESBABBAGEMAN ***(which is a valid predicate)***

But if we also wanted to represent

(ii) James Gosling *is a* man.

we would have to write something such as:

JAMESGOSLINGMAN ***(which is a valid predicate)***

- The above two are totally separate assertions and we are unable to draw any conclusions about similarities between *Charles Babbage* and *James Gosling.*
- So we need to rewrite them as follows:

  MAN(CHARLESBABBAGE)

  MAN(JAMESGOSLING)
- The structure of representation reflects the structure of knowledge itself. But to do that, we need to able to use predicates applied to arguments.

  (iii) All men are mortal.

  We could represent this as:

  ∀x : MAN(x) → *MORTAL(x)*
- The above statement fails to capture the relationship between any individual being a man and that individual being a mortal.
- To do that, we really need variables and quantification unless we are willing to write separate statements about the mortality of every known man.
- First-order predicate logic as a way of representing knowledge because it permits representations of things that cannot reasonably be represented in prepositional logic.
- In predicate logic, we can represent real-world facts as *statements* written as wff's.
- If we use logical statements as a way of representing knowledge, then we have available a good way of reasoning with that knowledge.
- Determining the validity of a proposition in propositional logic is straightforward, although it may be computationally hard. So before we adopt predicate logic as a good medium for representing knowledge, we need to ask whether it also provides a good way of reasoning with the knowledge.
- Predicate logic provides a way of deducing new statements from old ones, but it does not possess a decision procedure like propositional logic.

- Basically two ways to answer a question in knowledge base.
  1. Chaining
     (i) Forward Chaining
     (ii) Backward Chaining
  2. Resolution
- In Forward Chaining, we will have different facts to answer a question. (Facts → Goal)
- In Backward Chaining, start with question (GOAL), keep on substituting different facts to support answer with available facts. (i.e. In Backward Chaining replace right hand side rue with left hand side rule. So to solve / prove the given fact, we need to search every time in the right hand side)
- Chaining work on predicates.

Example 1: Translate the following facts to predicate logic and answer what course would Ramu like?

1. Ramu like only easy courses.
2. Science courses are hard.
3. All the courses in maths department are easy.
4. MA4501 is maths course.

First translate the above facts to predicate logic as follows:

1. Ramu like only easy courses

$\forall x : \text{Course}(x) \wedge \text{Easy}(x) \rightarrow \text{Like}(\text{Ramu}, x)$

2. Science courses are hard.

$\forall x : \text{Course}(x) \wedge \text{Science}(x) \rightarrow \text{Hard}(x)$

3. All the courses in maths department are easy.

$\forall x : \text{Course}(x) \wedge \text{Maths}(x) \rightarrow \text{Easy}(x)$

4. MA4501 is maths course.

$\text{Maths}(\text{MA4501}) \wedge \text{Course}(\text{MA4501})$

Question:

What course would Ramu like?

$\text{Like}(\text{Ramu}, x) \wedge \text{Course}(x)$

↓ (1)

Course(x) ∧ Easy(x) ∧ Course(x)

↓

Course(x) ∧ Easy(x)

↓ (3)

Course(x) ∧ Course(x) ∧ Maths(x)

↓

Course(x) ∧ Maths(x)

↓ (if x = MA4501)

(4) is TRUE

So answer is Ramu will like **MA501** course.

Example 2: Translate the following facts to predicate logic and prove Jhon likes peanuts using Backward Chaining.

1. John likes all kinds of foods.
2. Apples are food.
3. Chicken is food.
4. Anything anyone eats and is not kill by is food.
5. Bill eats peanuts and is still alive.
6. She eats everything Bill eats.

First translate the above facts to predicate logic as follows:

1. John likes all kinds of foods.

∀x **:** food(x) → likes(john, food)

2. Apples are food.

food(Apples)

3. Chicken is food.

food(Chicken)

4. Anything anyone eats and is not kill by is food.

∀x : ∀y : eats(x, y) ∧ ~killed(x, y) → food(y)

Here x is treated as anyone (i.e. person)

y is treated as food item.

5. Bill eats peanuts and is still alive.

eats(Bill, Peanuts) ∧ alive(Bill, Peanuts)

6. She eats everything Bill eats.

∀x **:** eats(Bill, x) → eats(She, x)

Add one more fact.

7. (a) ∀x : ∀y **:** alive(x, y) → ~killed(x, y)

(b) ∀x : ∀y **:** killed(x, y) → ~alive(x, y)

Now prove Jhon likes peanuts using Backward Chaining.

likes(John, Peanuts)

↓ (1) x = Peanuts

food(Peanuts)

↓ (4) x = x, y = Peanuts

∀x : eats(x, Peanuts) ∧ ~killed(x, Peanuts)

↓ (7(a)) y = Peanuts

∀x : eats(x, Peanuts) ∧ alive(x, Peanuts)

↓ (5) x = Bill

TRUE

Example 3: Use the following sentences and answer the question was Marcus loyal to Caesar?

1. Marcus was a man.
2. Marcus was a Pompeian.
3. All Pompeians were Romans.
4. Caesar was a ruler.
5. All Romans were either loyal to Caesar or hated him.
6. Everyone is loyal to someone.
7. People only try to assassinate rulers they are not loyal to.
8. Marcus tried to assassinate Caesar.

The facts described by these sentences can be represented as a set of wff's in predicate logic as follows:

1. Marcus was a man.

(Here General word is **Man**, Specific value is **Marcus**. So predicate name is General word)

man(Marcus)

2. Marcus was a Pompeian.

(Here General word is Pompeian (is a kind of tribe), Specific value is **Marcus**. So predicate name is General word)

Pompeian(Marcus)

3. All Pompeians were Romans.

$\forall x : Pompeian(x) \rightarrow Roman(x)$

4. Caesar was a ruler.

(Here General word is **ruler**, Specific value is Caesar. So predicate name is General word)

ruler(Caesar)

5. All Romans were either loyal to Caesar or hated him.

$\forall x: Roman(x) \rightarrow loyalto(x, Caesar) \quad V \quad hate(x, Caesar)$

6. Everyone is loyal to someone.

(**Everyone** require one variable and **someone** require another variable)

$\forall x : \exists y : loyalto(x, y)$

7. People only try to assassinate rulers they are not loyal to.

(Here General word is **People**, assassinate (i.e. kill), ruler, loyal to)

for example x is a people, y is a ruler

$\forall x : \forall y : people(x) \wedge ruler(y) \wedge tryassassinate(x, y) \rightarrow \sim loyalto(x, y)$

Meaning as follows;

(if **x** is people and **y** is ruler And **x** is try to assassinate **y** then **x** is not loyal to **y**)

8. Marcus tried to assassinate Caesar.

tryassassinate(Marcus, Caesar)

Question: Was Marcus loyal to Caesar?

loyalto(Marcus, Caesar)

Here Marcus was not loyal to Caesar, because Marcus tried to assassinate Caesar. So consider

~loyalto(Marcus, Caesar)

↓ (7) x = Marcus, y = Caesar

people(Marcus) ∧ ruler(Caesar) ∧ tryassassinate(Marcus, Caesar)

↓ (4) **TRUE**

people(Marcus) ∧ tryassassinate(Marcus, Caesar)

↓ (8) **TRUE**

people(Marcus)

- Here we don't have this predicate. But we have predicate as man(Marcus).
- But we know Man is a people. So we need to include another fact as follows:

9. All men's are peoples.

$\forall x : man(x) \rightarrow people(x)$

people(Marcus)

↓ (9) x = Marcus

man(Marcus)

↓ (1) **TRUE**

So finally ~loyalto(Marcus, Caesar) is **TRUE.**

∴ Was Marcus loyal to Caesar? **NO**

2.2 Representing *Instance* and *Isa* Relationships

- ***instance*** relationship used to express class membership (i.e. object to class relationship) and ***isa*** relationship used to express class inclusion (i.e. class to class relationship).
- Figure 2 shows the first five sentences represented in logic in three different ways.
- The first part of the figure contains the representations contains class membership is represented with unary predicates (such as *Roman),* each of which corresponds to a class. Asserting that *P(x)* is true is equivalent to asserting that x is an instance (or element) of *P*.
- The second part of the figure contains representations that use the *instance* predicate explicitly.

- The predicate *instance* is a binary one, whose first argument is an object and whose second argument is a class to which the object belongs.
- The third part contains representations that use both the *instance* and *isa* predicates explicitly. The use of the *isa* predicate simplifies the representation of sentence 3, but it requires that one additional axiom (shown here as number 6) be provided. This additional axiom describes how an *instance* relation and *an isa* relation can be combined to derive a new *instance* relation.

1. man(Marcus)
2. Pompeian(Marcus)
3. $\forall x$: Pompeian(x) $\rightarrow$Roman(x)
4. ruler(Caesar)
5. $\forall x$: Roman(x) $\rightarrow$ loyalto(x, Caesar) V hate(x, Caesar)

1. instance(Marcus, man)
2. instance(Marcus, Pompeian)
3. $\forall x$: instance(x, Pompeian) $\rightarrow$ instance(x, Roman)
4. instance(Caesar, ruler)
5. $\forall x$: instance(x, Roman) $\rightarrow$ loyalto(x, Caesar) V hate(x, Caesar)

1. instance(Marcus, man)
2. instance(Marcus, Pompeian)
3. isa(Pompeian, Roman)
4. instance(Caesar, ruler)
5. $\forall x$: instance(x, Roman) $\rightarrow$ loyalto(x, Caesar) V hate(x, Caesar)
6. $\forall x$: $\forall y$: $\forall z$: instance(x, y) $\wedge$ isa(y, z) $\rightarrow$ instance(x, z)

Figure 2: *Three Ways of Representing Class Membership*

Module 2: Semantic nets- frames and inheritance

3. SEMANTIC NETS

- The main idea behind semantic nets is that the meaning of a concept comes from the ways in which it is connected to other concepts.
- In a semantic net, information is represented as a set of nodes connected to each other by a set of labeled arcs, which represent relationships among the nodes.
- A fragment of a typical semantic net is shown in Figure 3.

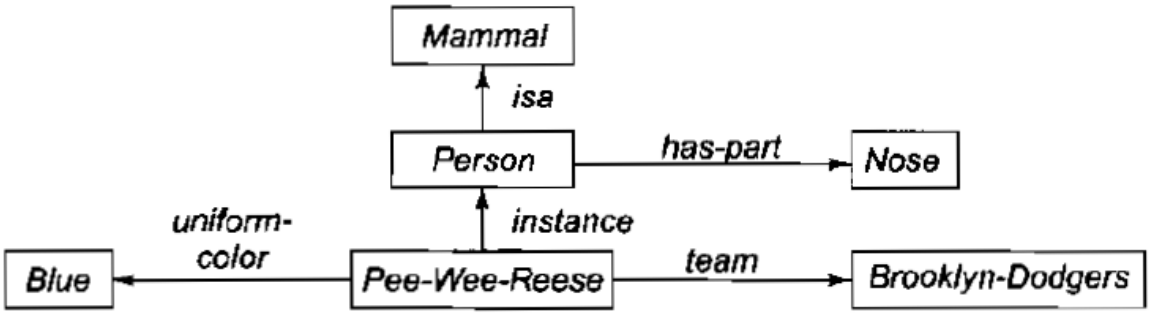


Figure 3: A Semantic Network

- This network contains examples of both the ***isa*** and ***instance*** relations, as well as some other, more domain-specific relations like **team** and **uniform-color**.
- In this network, we could use inheritance to derive the additional relation

  has part (Pee-Wee-Reese, Nose)
- The above semantic network represents the following statements.
  1. Person is a mammal.
  2. Pee-Wee-Reese is instance of Person.
  3. Person has a part Nose.
  4. Pee-Wee-Reese uniform color is Blue.
  5. Pee-Wee-Reese team name is Brooklyn-Dodgers.
- Various mechanisms of semantic nets as follows.

3.1 Intersection Search

- One of the early ways that semantic nets were used was to find relationships among objects by spreading activation out from each of two nodes and seeing where the activation met. This process is called *intersection search*.

- Using this process, it is possible to use the network of Figure 3 to answer questions such as "What is the connection between the Brooklyn Dodgers and blue?"

3.2 Representing Nonbinary Predicates

- Semantic nets are a natural way to represent relationships that would appear as ground instances of binary predicates in predicate logic.
- For example, some of the arcs from Figure 3 could be represented in logic as

  isa(Person, Mammal)

  instance(Pee-Wee-Reese, Person)

  team(Pee-Wee-Reese, Brooklyn-Dodgers)

  uniform-color(Pee-Wee-Reese, Blue)
- But the knowledge expressed by predicates of other arities can also be expressed in semantic nets. For example, many unary predicates in logic can be thought of as binary predicates using some very general-purpose predicates, such as ***isa*** and ***instance***. So, for example,

  man(Marcus)

  could be rewritten as

  instance(Marcus, Man)

  thereby making it easy to represent in a semantic net.
- Three or more place predicates can also be converted to a binary form by creating one new object representing the entire predicate statement and then introducing binary predicates to describe the relationship to this new object of each of the original arguments. For example, suppose we know that

  score(Cubs, Dodgers, 5-3)
- This can be represented in a semantic net by creating a node to represent the specific game and then relating each of the three pieces of information to it. Doing this produces the network shown in Figure 4.



Figure 4: A Semantic Net for an n-Place Predicate

- This technique is particularly useful for representing the contents of a typical declarative sentence that describes several aspects of a particular event.
- The sentence ***John gave the book to Mary*** could be represented by the network shown in Figure 5.

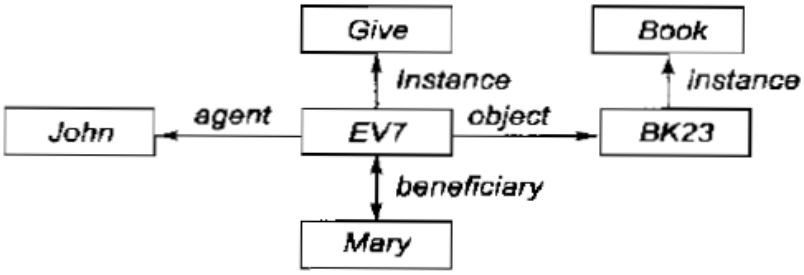
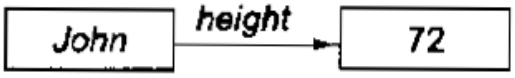


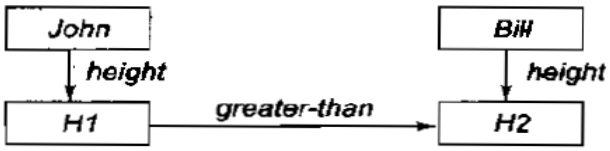
Figure 5: *A Semantic Net Representing a Sentence*

3.3 Making Some Important Distinctions

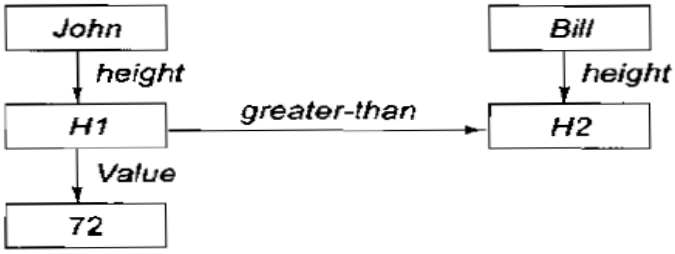
- In the networks we have described so far, we have glossed over some distinctions that are important in reasoning. For example, there should be a difference between a link that defines a new entity and one that relates two existing entities.
- Consider the net



- Both nodes represent objects that exist independently of their relationship to each other. But now suppose we want to represent the fact that John is taller than Bill, using the net



- The nodes H1 and H2 are new concepts representing John's height and Bill's height, respectively. They are defined by their relationships to the nodes *John* and *Bill.*
- Using these defined concepts, it is possible to represent such facts as that John's height increased, which we could not do before. (The number 72 increased?).
- Sometimes it is useful to introduce the arc *value* to make this distinction clear. Thus we might use the following net to represent the fact that John is 6 feet tall and that he is taller than Bill:



- The procedures that operate on nets such as this can exploit the fact that some arcs, such as *height,* define new entities, while others, such as *greater-than* and *value,* merely describe relationships among existing entities.

3.4 Partitioned Semantic Nets

- Suppose we want to represent simple quantified expressions in semantic nets. One way to do this is *to partition* the semantic net into a hierarchical set of *spaces,* each of which corresponds to the scope of one or more variables.
- To see how Partitioned Semantic Nets works, consider first the simple net shown in Figure 6(a). This net corresponds to the statement

  The dog bit the mail carrier

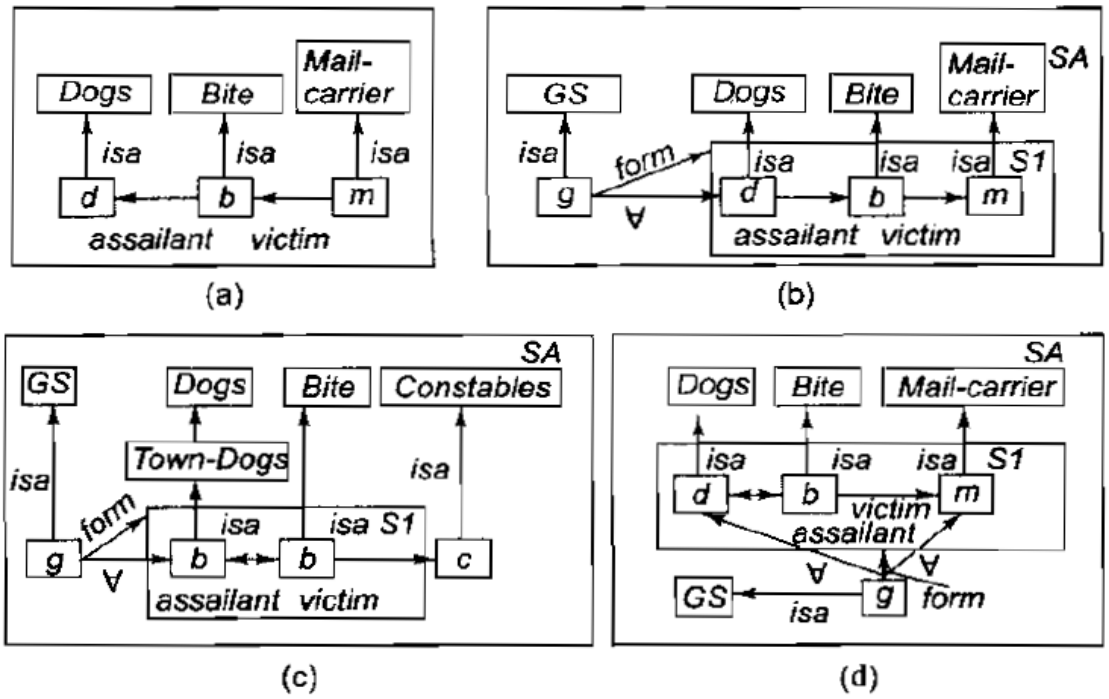


Figure 6: *Using Partitioned Semantic Nets*

- The nodes *Dog's, Bite,* and *Mail-Carrier* represent the classes of dogs, bitings, and mail carriers, respectively, while the nodes *d, b,* and *m* represent a particular dog, a particular biting, and a particular mail carrier.
- This fact can easily be represented by a single net with no partitioning. But now suppose that we want to represent the fact

Every dog has bitten a mail carrier, or, in logic:

$$\forall x: Dog(x) \rightarrow \exists y: Mail\text{-}Carrier\ (y) \wedge Bite(x, y)$$

- To represent this fact, it is necessary to encode the scope of the universally quantified variable x. This can be done using partitioning as shown in Figure 6(b).
- The node *g* stands for the assertion given above. Node *g* is an instance of the special class *GS* of general statements about the world (i.e., those with universal quantifiers).
- Every element of *GS* has at least two attributes: *a form*, which states the relation that is being asserted, and one or more $\forall$ connections one for each of the universally quantified variables.
- In this example, there is only one such variable *d*, which can stand for any element of the class *Dogs*. The other two variables in the form, *b* and *m*, are understood to be existentially quantified. In other words, for every dog *d*, there exists a biting event *b*, and a mail carrier m, such that *d* is the assailant of *b* and *m* is the victim.
- To see how partitioning makes variable quantification explicit, consider next the similar sentence: Every dog in town has bitten the constable.
- The representation of this sentence is shown in Figure 6(c). In this net, the node ***c*** representing the victim lies outside the form of the general statement. Thus it is not viewed as an existentially quantified variable whose value may depend on the value of ***d***. Instead it is interpreted as standing for a specific entity (in this case, a particular constable), just as do other nodes in a standard, non partitioned net.
- Figure 6(d) shows how yet another similar sentence:

  Every dog has bitten every mail carrier

  would be represented. In this case, ***g*** has two $\forall$ links, one pointing to ***d***, which represents any dog, and one pointing to ***m***, representing any mail carrier.
- The spaces of a partitioned semantic net are related to each other by an inclusion hierarchy. For example, in Figure 6(d), space **S1** is included in space **SA**.
- Whenever a search process operates in a partitioned semantic net, it can explore nodes and arcs in the space from which it starts and in other spaces that contain the starting point, but it cannot go downward, except in special circumstances, such as when *a form* arc is being traversed.

4. FRAMES

- The idea of a semantic net started out simply as a way to represent labeled connections among entities.
- The problem-solving tasks that the representation must support, the representation itself necessarily begins to become more complex.

- In particular it becomes useful to assign more structure to nodes as well as to links.
- A frame is a collection of attributes (usually called slots) and associated values (and possibly constraints on values) that describe some entity in the world.
- Sometimes a frame describes an entity in some absolute sense; sometimes it represents the entity from a particular point of view.
- A single frame taken alone is rarely useful. Instead, we build frame systems out of collections of frames that are connected to each other by good quality of the fact that the value of an attribute of one frame may be another frame.

Person		
isa	:	Mammal
cardinality	:	6,000,000,000
* handed	:	Right
Adult-Male		
isa	:	Person
cardinality	:	2,000,000,000
* height	:	5-10
ML-Baseball-Player		
isa	:	Adult-Male
cardinality	:	624
*height	:	6-1
* bats	:	equal to handed
* batting-average	:	.252
* team	:	
* uniform-color	:	Red
Fielder		
isa	:	ML-Baseball-Player
cardinality	:	376
*batting-average:		.262
Pee-Wee-Reese		
instance	:	Fielder
height	:	5-10
bats	:	Right
batting-average	:	.309
team	:	Brooklyn-Dodgers
uniform-color	:	Blue
ML-Baseball-Team		
isa	:	Team
cardinality	:	26
* team-size	:	24

* manager	:	Ramu
Brooklyn-Dodgers		
instance	:	ML-Baseball-Team
team-size	:	24
manager	:	Leo-Durocher
players	:	{Pee-Wee-Reese,...)

Figure 7: A Simplified Frame System

- A class represents a set, there are two kinds of attributes that can be associated with it. There are attributes about the set itself, and there are attributes that are to be inherited by each element of the set. We indicate the difference between these two by prefixing with an asterisk (*).

5. INHERITANCE

- To support flexible representations of knowledge about the world, it is necessary to allow the hierarchy to be an arbitrary Directed Acyclic Graph (DAG).
- Acyclic graphs are adequate because *isa* corresponds to the subset relation.
- Hierarchies that are not trees are called *tangled hierarchies.*
- Consider the example shown in Figure 8 (a). We want to decide whether *Fifi* can fly. The correct answer is no.

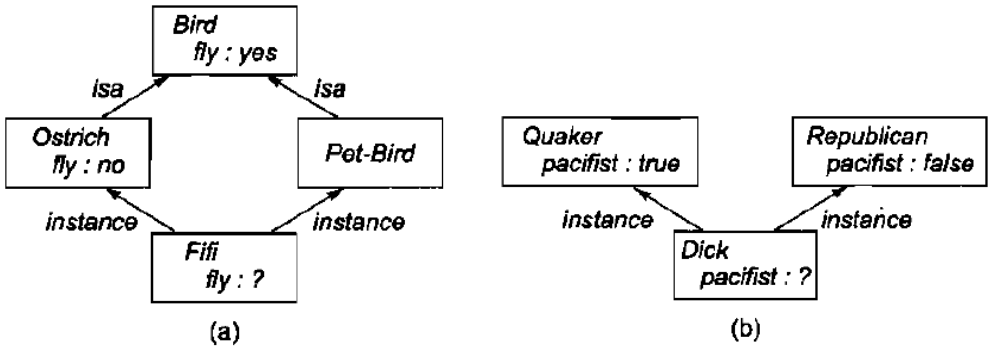


Figure 8: Tangled Hierarchies

- Although birds in general can fly, the subset of birds, ostriches, does not. Although the class *Pet-Bird* provides a path from *Fifi* to *Bird* and thus to the answer that *Fifi* can fly, it provides no information that conflicts with the special case knowledge associated with the class *Ostrich,* so it should have no affect on the answer.
- To handle this case correctly, we need an algorithm for traversing the *isa* hierarchy that guarantees that specific knowledge will always dominate more general facts.
- Figure 8 (b), determining whether Dick is a pacifist. Again, we must traverse multiple *instance* links, and more than one answer can be found along the paths. But in this case, there is no well-founded basis for choosing one answer over the other.

- The classes that are associated with the candidate answers are inadequate with each other in the partial ordering that is defined by the DAG formed by the *isa* hierarchy.
- One possible basis for a new inheritance algorithm is path length. This can be implemented by executing a breadth-first search, starting with the frame for which a slot value is needed. Follow its *instance* links, then follow *isa* links upward.
- If a path produces a value, it can be terminated, as can all other paths once their length exceeds that of the successful path. This algorithm works for both of the examples in Figure 8. In (a), it finds a value at *Ostrich.* It continues the other path to the same length *(Per-Bird), fails* to find any other answers, and then halts. In the case of (b), it finds two competing answers at the same level, so it can report the contradiction.

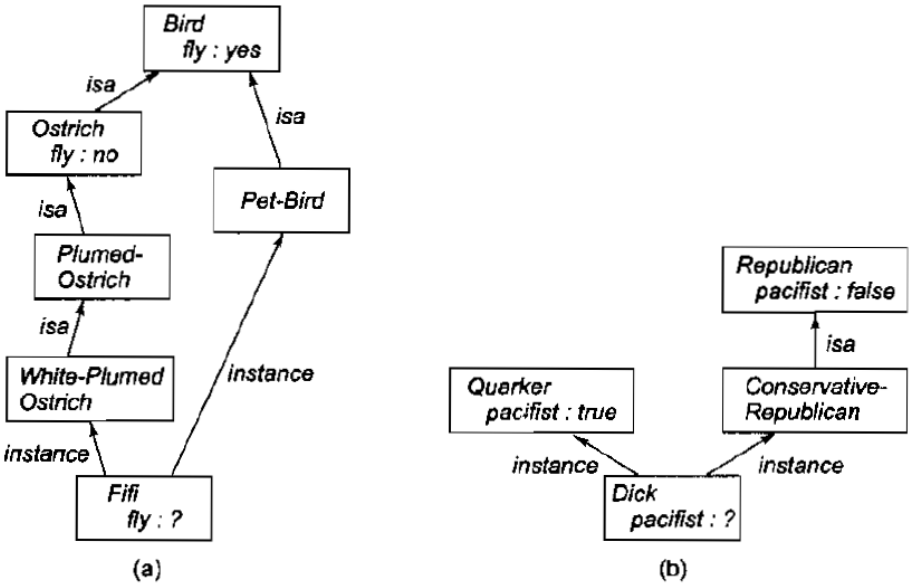


Figure 9: *More Tangled Hierarchies*

- But now consider the examples shown in Figure 9. In the case of (a), our new algorithm reaches *Bird (via Pet-Bird)* before it reaches *Ostrich.* So it reports that *Fifi* can fly.
- In the case of (b), the algorithm reaches *Quaker* and stops without noticing a contradiction. The problem is that path length **does not** always correspond to the level of generality of a class. Sometimes what it really corresponds to is the degree of elaboration of classes in the knowledge base.
- If some regions of the knowledge base have been elaborated more fully than others, then their paths will tend to be longer. But this should not influence the result of inheritance if no new information about the desired attribute has been added.
- The solution to this problem is to base inheritance algorithm not on path length but on the notion of *inferential distance,* which can be defined as follows:

- *Class1* is closer to *Class2* than to *Class3*, if and only if *Class1* has an inference path to *Class3* through *Class2* (in other words, *Class2* is between *Class1* and *Class3).*
- Notice that inferential distance defines only a partial ordering. Some classes are inadequate with each other under it.
- We can now define the result of inheritance as follows: The set of competing values for a slot ***S*** in a frame ***F*** contains all those values that

  1. Can be derived from some frame X that is above *F* in the *isa* hierarchy

  2. Are not contradicted by some frame Y that has a shorter inferential distance to *F* than *X* does.

Algorithm: Property Inheritance

To retrieve a value V for slot S of an instance *F* do:

1. Set *CANDIDATES* to empty.

2. Do breadth-first or depth-first search up the *isa* hierarchy from ***F*** following all ***instance*** and ***isa*** links. At each step, see if a value for ***S*** or one **F** its generalizations is stored.

   (a) If a value is found, add it to *CANDIDATES* and terminate that branch of the search.

   (b) If no value is found but there are *instance* or *isa* links upward, follow them.

   (c) Otherwise, terminate the branch.

3. For each element **C** of *CANDIDATES* do:

   (a) See if there is any other element of *CANDIDATES* that was derived from a class closer to ***F*** than the class from which ***C*** came.

   (b) If there is, then, remove ***C*** from *CANDIDATES.*

4. Check the cardinality of *CANDIDATES:*

   (a) If it is 0, then report that no value was found.

   (b) If it is 1, then return the single element of *CANDIDATES* as **V**.

   (c) If it is greater than 1, report a contradiction.

- This algorithm is guaranteed to terminate because the *isa* hierarchy is represented as an acyclic graph.

6. REPRESENTING KNOWLEDGE USING RULES

- Rules to encode knowledge is a particularly important issue since rule-based reasoning systems has played a very important role in the evolution of AI.

6.1 Procedural versus Declarative Knowledge

- A *declarative representation* is one in which knowledge is specified, but the use to which that knowledge is not given.
- To use a declarative representation, we must provide it with a program that specifies what is to be done to the knowledge and how.
- For example, a set of logical assertions can be combined with a resolution theorem prover to give a complete program for solving problems.
- There is a different way, though, in which logical assertions can be viewed, namely as a *program,* rather than as *data* to a program.
- A *procedural representation* is one in which the control information that is necessary to use the knowledge is considered to be embedded in the knowledge itself.
- To use a procedural representation, we need to augment it with an interpreter that follows the instructions given in the knowledge.
- Actually, viewing logical assertions as code is not a very radical idea, given that all programs are really data to other programs that interpret (or compile) and execute them.
- The real difference between the declarative and the procedural views of knowledge lies in where control information resides. For example, consider the knowledge base:

  man(Marcus)

  man(Caesar)

  person(Cleopatra)

  $\forall x : man(x) \rightarrow person(x)$
- Now consider trying to extract from this knowledge base the answer to the question

  $\exists y$*: person(y)*
- We want to bind *y* to a particular value for which *person* is true. Our knowledge base justifies any of the following answers:

  y = Marcus

  y = Caesar

  y = Cleopatra
- Because there is more than one value that satisfies the predicate, but only one value is needed, the answer to the question will depend on the order in which the assertions are examined during the search for a response.
- If we view the assertions as declarative, then they do not themselves say anything about how they will be examined. If we view them as procedural, then they do.

- To see clearly the difference between declarative and procedural representations, consider the following assertions:

  man(Marcus)

  man(Caesar)

  $\forall x : man(x) \rightarrow person(x)$

  person(Cleopatra)

- Viewed declaratively, this is the same knowledge base that we had before. All the same answers are supported by the system and no one of them is explicitly selected.
- But viewed procedurally, and using the control model we used to get *Cleopatra* as our answer before, this is a different knowledge base since now the answer to our question is *Marcus.* This happens because the first statement that can achieve the *person* goal is the inference rule $\forall x : man(x) \rightarrow person(x)$.
- This rule sets up a subgoal to find a man. Again the statements are examined from the beginning, and now *Marcus* is found to satisfy the subgoal and thus also the goal. So *Marcus* is reported as the answer.
- It is important to keep in mind that although we have said that a procedural representation encodes control information in the knowledge base, it does so only to the extent that the interpreter for the knowledge base recognizes that control information.
- So we could have gotten a different answer to the *person* question by leaving our original knowledge base intact and changing the interpreter so that it examines statements from last to first.

6.1.1 Difference the Procedural and Declarative Knowledge

PROCEDURAL KNOWLEDGE	DECLARATIVE KNOWLEDGE
It is also known as Interpretive knowledge.	It is also known as Descriptive knowledge.
Procedural Knowledge means how a particular thing can be accomplished	While Declarative Knowledge means basic knowledge about something.
Procedural Knowledge is generally not used means it is not more popular.	Declarative Knowledge is more popular.
Procedural Knowledge can't be easily communicate.	Declarative Knowledge can be easily communicate
Procedural Knowledge is generally process oriented in nature	Declarative Knowledge is data oriented in nature.
In Procedural Knowledge debugging and validation is not easy.	In Declarative Knowledge debugging and validation is easy.

6.2 Logic Programming

- Logic programming is a programming language paradigm in which logical assertions are viewed as programs.

- There are several logic programming systems in use today, the most popular of which is PROLOG.
- A PROLOG program is described as a series of logical assertions, each of which is a Horn clause.
- A Horn clause is a clause that has at most one positive literal.

  For example p, ~p V q, and p → q are all Horn clauses.
- The control structure of PROLOG interpreter is backward reasoning is applied to try to prove the goal given the assertions in the program. The program is read top to bottom, left to right and search is performed depth-first with backtracking.

∀x : pet(x) ∧ small(x) → apartmentpet(x)

∀x : cat(x) V dog(x) → Pet(x)

∀x : poodle(x) → dog(x) ∧ small(x)

poodle(fluffy)

A Representation in Logic

apartmentpet (X) :- pet(X), small (X)

pet(X) :- cat (X)

pet(X) :- dog (X)

dog(X) :- poodle (X)

small(X) :- poodle (X)

poodle(fluffy)

A Representation in PROLOG

Figure 10: A Declarative and a Procedural Representation

- Figure 10 shows an example of a simple knowledge base represented in standard logical notation and then in PROLOG.
- Both of these representations contain two types of statements, facts, which contain only constants (i.e., no variables) and rules, which do contain variables. Facts represent statements about specific objects. Rules represent statements about classes of objects.

Logic Programming	PROLOG

Variables are explicitly quantified	Quantification is provided implicitly by the way the variables are interpreted. Variables begin with upper case letters Constants begin with lower case letters or numbers.
There are explicit symbols for and (∧) and or (v)	There is an explicit symbol for and (**,**) but there is none for **or**. Instead, disjunction must be represented as a list of alternative statements
Implications of the form "p implies q" are written as p→q	"p implies q" implication is written "backward" as q:- p.

6.3 Forward versus Backward Reasoning

- The object of a search procedure is to discover a path through a problem space from an initial configuration to a goal state.
- While PROLOG only searches from a goal state, there are actually two directions in which such a search could proceed:

  Forward, from the start states.

  Backward, from the goal states.

6.3.1 Reason forward from the initial states

- Begin building a tree of move sequences that might be solutions by starting with the initial configuration(s) at the root of the tree.
- Generate the next level of the tree by finding all the rules whose left sides match the root node and using their right sides to create the new configurations.
- Generate the next level by taking each node generated at the previous level and applying to it all of the rules whose left sides match it.
- Continue until a configuration that matches the goal state is generated.

6.3.2 Reason backward from the goal states

- Begin building a tree of move sequences that might be solutions by starting with the goal configuration(s) at the root of the tree.
- Generate the next level of the tree by finding all the rules whose right sides match the root node. These are all the rules that, if only we could apply them, would generate the state we want.
- Use the left sides of the rules to generate the nodes at this second level of the tree.

- Generate the next level of the tree by taking each node at the previous level and finding all the rules whose right sides match it. Then use the corresponding left sides to generate the new nodes.
- Continue until a node that matches the initial state is generated. This method of reasoning backward from the desired final state is often called goal-directed reasoning.

- Four factors influence, the question of whether it is better to reason forward or backward:
  1. Move from the smaller set of states to the larger set of states.
  2. Proceed in the direction with the lower branching factor (i.e.: the average number of nodes that can be reached directly from a single node).
  3. Proceed in the direction that corresponds more closely with the way the user will think.
  4. Proceed in the direction that corresponds more closely with the way the problem solving episodes will be triggered

- To encode knowledge about reasoning, we need two kinds of rules:
  1. Forward rules, which encode knowledge about how to respond to certain input configurations.
  2. Backward rules, which encode knowledge about how to achieve particular goals.

6.3.3 Difference between backward chaining and forward chaining

Forward Chaining	Backward Chaining
It starts from known facts and applies inference rule to extract more data unit it reaches to the goal.	It starts from the goal and works backward through inference rules to find the required facts that support the goal.
It is a bottom-up approach	It is a top-down approach
It is known as data-driven inference technique as we reach to the goal using the available data.	It is known as goal-driven technique as we start from the goal and divide into sub-goal to extract the facts.
It applies a breadth-first search strategy	It applies a depth-first search strategy.
It tests for all the available rules	It tests only for few required rules.

It is suitable for the planning, monitoring, control, and interpretation application.	It is suitable for diagnostic, prescription, and debugging application
It can generate an infinite number of possible conclusions	It generates a finite number of possible conclusions
It operates in the forward direction	It operates in the backward direction

Module 3: Reasoning under uncertainty

7. PROBABILITY

- Probability is defined as the chance of occurrences of an event. Generally, the possibility of analyzing the occurrence of any event with respect to previous data is called probability.
- For example, if a fair coin is tossed, what is the chance that it lands on the head? These types of questions are answered under probability.
- The value of probability always between 0 and 1 that represent ideal uncertainties.
- Axioms of probability as follows:

  $0 \leq P(A) \leq 1$, where P(A) is the probability of an event A.

  P(A) = 0, indicates total uncertainty in an event A.

  P(A) = 1, indicates total certainty in an event A.

- Probability of an uncertain event can be calculated using the following formula.

$$\text{Probability of occurrence} = \frac{Number\ of\ desired\ outcomes}{Total\ number\ of\ outcomes}$$

- P(~A) = Probability of a not happening event.
- P(~A) + P(A) = 1

7.1 Probabilistic Reasoning

- Probabilistic Reasoning is a way of knowledge representation where we apply the concept of probability to indicate the uncertainty in the knowledge.

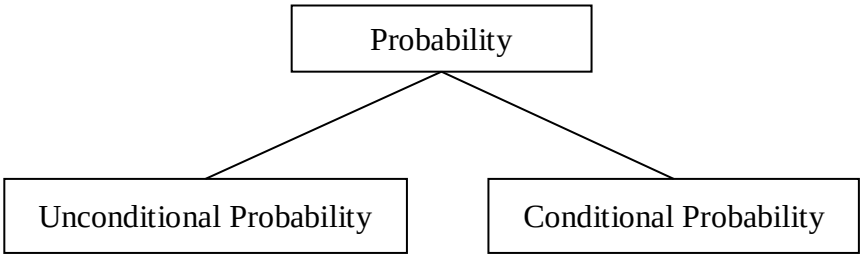
- In the real world, there are lots of scenarios, where the certainty of something is not confirmed, such as

  It will rain today

  Behaviour of someone for some situations

  A match between two teams or two players

- These are probable sentences for which we can assume that it will happen but not sure about it, so here use probabilistic reasoning.



- Two events are said to be **independent** if one event occurring does not affect the probability that the other event will occur.
- However, if one event occurring, affect the probability that the other event will occur, the two events are said to be **dependent**.
- If events are independent, then the probability of some event B is not dependent on what happens with event A.
- A conditional probability, therefore, relates to those events that are dependent on one another.

7.2 Unconditional or prior probabilities

- Unconditional probability refers an event will take place irrespective of whether any other events have taken place or any other conditions are present.
- Uncertainty is defined as the lack of the knowledge that would enable us to reach a perfectly reliable conclusion.

7.3 Conditional or posterior probability

- Conditional probability is defined as an event or outcome occurring, based on the occurrence of a previous event or outcome.
- Conditional probability is calculated by multiplying the probability of the preceding event by the updated probability of the succeeding, or conditional, event.
- Conditional Probability formula as follows:

  $P(B|A) = P(A \cap B) / P(A)$

  which holds whenever $P(A) > 0$.

  Where

P = Probability

A = Event A

B = Event B

in another form. i.e. product rule.

$P(A \wedge B) = P(B|A)\ P(A)$

8. BAYES' PROBABILISTIC INTERFERENCES

- Bayes theorem was named after the British mathematician Thomos Bayes.
- Bayes theorem is also known as Bayes rule, Bayes law, or Bayesian reasoning, which determines the probability of an event with uncertainty knowledge.
- Bayes theorem allows updating the probability prediction of an event by observing new information of the real world.
- The product rule can actually be written in two forms:

$$P(a \wedge b) = P(a \mid b)\ P(b) \quad \text{and} \quad P(a \wedge b) = P(b \mid a)\ P(a)\ .$$

- Equating the two right-hand sides and dividing by P(a), we get

$$P(b \mid a) = \frac{P(a \mid b)P(b)}{P(a)}$$

- This equation is known as **Bayes' rule** (also Bayes' law or Bayes' theorem). This simple equation underlies most modern AI systems for probabilistic inference.
- The more general case of Bayes' rule for multi-valued variables can be written in the **P** notation as follows:

$$\mathbf{P}(Y \mid X) = \frac{\mathbf{P}(X \mid Y)\mathbf{P}(Y)}{P(X)}$$

- To use a more general version conditionalized on some background evidence **e** as follows:

$$\mathbf{P}(Y \mid X, e) = \frac{\mathbf{P}(X \mid Y, e)\mathbf{P}(Y \mid \mathbf{e})}{\mathbf{P}(X \mid \mathbf{e})}$$

8.1 Applying Bayes' rule: The simple case

- Bayes' rule allows us to compute the single term P(b | a) in terms of three terms: P(a | b), P(b), and P(a). That seems like two steps backwards, but Bayes' rule is useful in practice because there are many cases where we do have good probability estimates for these three numbers and need to compute the fourth.
- We perceive as evidence the *effect* of some unknown *cause* and we would like to determine that cause. In that case, Bayes' rule becomes as follows:

$$\mathrm{P}(\text{cause} \mid effect) = \frac{\mathrm{P}(effect \mid \text{cause})\ \mathrm{P}(\text{cause})}{\mathrm{P}(effect)}$$

- The conditional probability P(*effect* | cause) quantifies the relationship in the **causal** direction, whereas P(cause | *effect*) describes the **diagnostic** direction.
- In a task such as medical diagnosis, we often have conditional probabilities on causal relationships, i.e., the doctor knows P(symptoms | disease) and want to derive a diagnosis, P(disease | symptoms).
- For example, a doctor knows that the disease meningitis causes the patient to have a stiff neck, say, 70% of the time.
- The doctor also knows some unconditional facts: the prior probability (Unconditional) that a patient has meningitis is 1/50,000, and the prior probability that any patient has a stiff neck is 1%.
- Letting **s** be the proposition that the patient has a stiff neck and **m** be the proposition that the patient has meningitis, we have

  $\mathrm{P}(s \mid m) = 0.7$

  $\mathrm{P}(m) = 1/50000$

  $\mathrm{P}(s) = 0.01$

  $$\mathrm{P}(m \mid s) = \frac{\mathrm{P}(s \mid m)\mathrm{P}(m)}{\mathrm{P}(s)} = \frac{0.7 \times 1/50000}{0.01} = 0.0014$$

- That is, we expect less than 1 in 700 patients with a stiff neck to have meningitis.
- Notice that even though a stiff neck is quite strongly indicated by meningitis (with probability 0.7), the probability of meningitis in the patient remains small.
- This is because the prior probability of stiff necks is much higher than that of meningitis.
- One can avoid assessing the prior probability (Unconditional probability) of the evidence (here, P(s)) by instead computing a posterior probability (Conditional probability) for each value of the query variable (here, m and ¬ m) and then

normalizing the results. The same process can be applied when using Bayes' rule. We have:

P(M | s) = α <P(s |m) P(m), P(s | ¬ m) P(¬ m)> .

- Thus, to use this approach we need to estimate P(s | ¬ m) instead of P(s).
- The general form of Bayes' rule with normalization is **P**(Y |X) = α **P**(X | Y)**P**(Y)
- Where α is the normalization constant needed to make the entries in **P**(Y |X) sum to 1.

8.2 Using Bayes' rule: Combining evidence

- We have seen that Bayes' rule can be useful for answering probabilistic queries conditioned on one piece of evidence - for example, the stiff neck.
- In particular, we have argued that probabilistic information is often available in the form P(*effect* | cause). What happens when we have two or more pieces of evidence?
- For example, what can a dentist conclude if catches in the tooth aching of a patient?

	toothache		~toothache	
	catch	~catch	catch	~catch
cavity	0.108	0.012	0.072	0.008
¬cavity	0.016	0.064	0.144	0.576

Figure 11: A full joint distribution for the Toothache, Cavity, Catch world

- **P**(Cavity | toothache ∧ catch) = α <0.108, 0.016> ≈ <0.871, 0.129> .
- We know, however, that such an approach does not scale up to larger numbers of variables.
- We can try using Bayes' rule to reformulate the problem:

  P(Cavity | toothache ∧ catch) = α **P**(toothache ∧ catch | Cavity) **P**(Cavity) .

- For this reformulation to work, we need to know the conditional probabilities of the conjunction ***toothache ∧catch*** for each value of Cavity. That might be feasible for just two evidence variables, but again it does not scale up.

- If there are **n** possible evidence variables (X rays, diet, oral hygiene, etc.), then there are $\mathbf{2^n}$ possible combinations of observed values for which we would need to know conditional probabilities.

9. DEMPSTER SHAFER THEORY

- Dempster-Shafer theory approach considers sets of propositions and assigns to each of them an interval [Belief, Plausibility] in which the degree of belief must lie.
- Belief (usually denoted Bel) measures the strength of the evidence in favor of a set of propositions. It ranges from **0** (indicating no evidence) to **1** (denoting certainty).
- Plausibility (Pl) is denned to be Pl(s) = 1 - Bel(~s). It also ranges from 0 to 1 and measures the extent to which evidence in favor of **~s** leaves room for belief in **s**.
- In particular, if we have certain evidence in favor of **~s**, then Bel(~s) will be **1** and Pl(s) will be **0**. This tells us that the only possible value for Bel(s) is also 0.
- For example, in a simplified diagnosis problem, Θ might consist of the set {All, Flu, Cold, Pneu}

  All: Allergy

  Flu: Flu

  Cold: Cold

  Pneu: Pneumonia
- Our goal is to attach some measure of belief to elements of Θ. However, not all evidence is directly supportive of individual elements. Often it supports sets of elements (i.e., subsets of Θ).
- For example, in our diagnosis problem, fever might support (Flu, Cold, Pneu).
- The key function we use is a probability density function, which we denote as **m**.
- The function **m** is defined not just for elements of **e** but for all subsets of it (including singleton subsets, which correspond to individual elements).
- Let us see how **m** works for our diagnosis problem. Assume that we have no information about how to choose among the four hypotheses when we start the diagnosis task. Then we define in as: { Θ } = (1.0)
- All other values of in are thus 0. Although this means that the actual value must be some one element All, Flu, Cold, or Pneu, we do not have any information that allows us to assign belief in any other way than to say that we are sure the answer is somewhere in the whole set.
- Suppose we acquire a piece of evidence that suggests (at a level of 0.6) that the correct diagnosis is in the set {Flu, Cold, Pneu}.

- Fever might be such a piece of evidence. We update **m** as follows:

  {Flu, Cold, Pneu) (0.6)

  { Θ } (0.4)

- At this point, we have assigned to the set {Flu, Cold, Pneu} the appropriate belief. The remainder of our belief still resides in the larger set Θ.

- Dempster's rule allows us to combine any two belief functions i.e. whether they represent multiple sources of evidence for a single hypothesis or multiple sources of evidence for different hypotheses.

- Suppose we are given two belief functions **m1** and **m2**, Let **X** be the set of subsets of Θ to which **m1** assigns a nonzero value and let **Y** be the corresponding set for **m2**. We define the combination **m3** of **m1** and **m2** to be

$$\mathbf{m_3(Z)} = \frac{\sum_{X \cap Y = Z} \mathbf{m_1(X) \cdot m_2(Y)}}{\mathbf{1 - \sum_{X \cap Y = \emptyset} m_1(X) \cdot m_2(Y)}}$$

- For example, suppose m1 corresponds to our belief after observing fever:

  (Flu, Cold, Pneu} (0.6)

  { Θ } (0.4)

- Suppose m2 corresponds to our belief after observing a runny nose:

  {All, Flu, Cold} (0.8)

  { Θ } (0.2)

- Then we can compute their combination **m3** using the following table, which we can derive using the numerator of the combination rule:

		{A, F, C}	(0.8)	Θ	(0.2)
{F, C, P}	(0.6)	{F, C}	(0.48)	{F, C, P}	(0.12)
Θ	(0.4)	{A, F, C}	(0.32)	Θ	(0.08)

- The four sets that are generated by taking all ways of intersecting an element of X and an element of Y are shown in the body of the table.

- The value of m3 that the combination rule associates with each of them is computed by multiplying the values of m1 and m2 associated with the elements from which they were derived.

QUESTION BANK

1 Mark: Objective Questions

Module 1: Knowledge representation and predicate logic

1. Which of the following statements correctly define knowledge representation in AI?

   (a) It is the way in which facts and information are stored in the storage system of the agent [ ]

   (b) It is the way in which we feed the knowledge in machine understandable form

   (c) We modify the knowledge and convert it into the format which is acceptable by the machine

   (d) All of the above

2. Translate the following statement into First Order Logic. [ ]

   "For every x, if a is x philosopher, then x is a scholar"

   (a) ∀ x: philosopher(x) → scholar(x) (b) ∃ x: philosopher(x) → scholar(x)

   (c) All of the mentioned (d) None of the mentioned

3. Knowledge and reasoning also play a crucial role in dealing with _____ environment. [ ]

   (a) Completely Observable (b) Partially Observable

   (c) Neither Completely nor Partially Observable

   (d) Only Completely and Partially Observable

Module 2: Semantic nets- frames and inheritance

4. Semantic Network represents _____ [ ]

   (a) Syntactic relation between concepts (b) Semantic relations between concepts

   (c) All of the mentioned (d) None of the mentioned

5. What are Semantic Networks? [ ]

   (a) A way of representing knowledge (b) Data Structure

   (c) Data Type (d) None of the mentioned

6. Graph used to represent semantic network is _____ [ ]

   (a) Undirected graph (b) Directed graph

   (c) Directed Acyclic graph (DAG) (d) Directed complete graph

7. The basic inference mechanism in semantic network is to follow the links between the nodes. [ ]

   (a) True (b) False

8. What is the frame? [ ]

   (a) A way of representing knowledge (b) Data Structure

   (c) Data Type (d) Structure

9. Frames in artificial intelligence is derived from semantic nets. [ ]

   (a) True (b) False

10. Select the extension of the semantic network? [ ]

    (a) Partitioned Networks (b) Rule-Based Expert Systems

    (c) Decision Tree Based networks (d) Expert Systems

11. Which of the following elements constitutes the frame structure? [ ]

    (a) Facts or Data (b) Procedures and default values

    (c) Frame names (d) Frame reference in hierarchy

12. Forward chaining systems are _____ where as backward chaining systems are _____ [ ]

    (a) Goal-driven, goal-driven (b) Goal-driven, data-driven

    (c) Data-driven, goal-driven (d) Data-driven, data-driven

Module 3: Reasoning under uncertainty

13. How many terms are required for building a baye's model? [ ]

(a) 1 (b) 2 (c) 3 (d) 4

14. Where does the bayes rule can be used _____? [ ]

(a) Solving queries (b) Increasing complexity

(c) Decreasing complexity (d) Answering probabilistic query

15. Among which of the following mentioned statements can the Bayesian probability be applied? [ ]

(i) In the cases, where we have one event

(ii) In the cases, where we have two events

(iii) In the cases, where we have three events

(iv) In the cases, where we have more than three events

(a) Only (iv) (b) (i), (ii), (iii) and (iv)

(c) (ii) and (iv) (d) Only (ii)

16. According to Dempster-Shafer theory for uncertainty management. [ ]

Where Bel(A) denotes Belief of event A. **(UGC NET - Dec 2019 Paper)**

(a) $Bel(A) + Bel(\neg A) \leq 1$ (b) $Bel(A) + Bel(\neg A) \geq 1$

(c) $Bel(A) + Bel(\neg A) = 1$ (d) $Bel(A) + Bel(\neg A) = 0$

17. Dempster-Shafer theory is commonly used in which of the following fields? [ ]

(a) Decision making (b) Problem Solving

(c) Output Analysis (d) Al of the mentioned

2 Marks: Short Answer Questions

Module 1: Knowledge representation and predicate logic

1. List different knowledge representation issues.
2. Specify how Forward Chaining will solve the given problem.
3. Mention procedure to solve the given problem using Backward Chaining.
4. Specify in which situation Instance and Isa Relationships will be used.

Module 2: Semantic nets- frames and inheritance

5. Draw a simple semantic net that incorporate isa and instance relations.
6. List different mechanisms of semantic nets.
7. Mention use of Partitioned Semantic nets.

8. Difference the Procedural and Declarative Knowledge.

Module 3: Reasoning under uncertainty

9. Discriminate conditional and unconditional probability.
10. Obtain Bayes rule from the product rule.

5 Marks: Descriptive Questions

Section – B

Module 1: Knowledge representation and predicate logic

1. Discuss various knowledge representation issues.
2. With an example describe representation of simple facts in logic.
3. Translate the following facts to predicate logic and answer what course would Hari like?
    1. Hari like only tough courses.
    2. Science courses are easy.
    3. All the courses in maths department are tough.
    4. CS4501 is maths course.

Module 2: Semantic nets- frames and inheritance

4. What is Semantic Network? Describe with an example.
5. Describe Partitioned Semantic Nets.
6. What is a frame? Describe a simplified frame system with an example.
7. Explain Tangled Hierarchies and More Tangled Hierarchies of inheritance mechanism.
8. Describe algorithm of Property Inheritance.
9. Examine Procedural versus Declarative Knowledge.

10. Examine role of Logic Programming in representing knowledge using rules.
11. Differentiate backward chaining and forward chaining.

Module 3: Reasoning under uncertainty

12. Explain how reasoning under uncertainty can be done with probability.
13. With an example describe how Bayes' rule can be applied to the simple case.
14. A doctor is aware that disease meningitis causes a patient to have a stiff neck, and it occurs 80% of the time. He is also aware of some more facts, which are given as follows: The Known probability that a patient has meningitis disease is 1/30,000. The Known probability that a patient has a stiff neck is 2%. What is the probability that a patient has diseases meningitis with a stiff neck?
15. With an example describe how Bayes' rule can be applied to the Combining evidence
16. Explain dempster shafer theory with an example.